

Chapter 3: Planning with Intent for Data Engineering

A Framework for Transforming Business Requests into Implementation-Ready Artifacts

⚡ QUICK START: Want Hands-On First?

If you prefer learning by doing, jump to **Section 3.22 (Hands-On Tutorial)**. You can run the agent pipeline against a sample project and return here to understand what's happening under the hood.

- **Quick path:** 3.1 → 3.2 → 3.13 → 3.22
- **Recommended path:** Read sequentially — the human workflow (3.3-4.10) explains what the agents automate

The agents are only as good as your ability to review their outputs. Understanding the human roles makes you a better reviewer.

Chapter Structure

This chapter is organized into two major sections:

Section	Parts	Focus	Pages
The Human Workflow	I, II, III	How humans plan data projects today	~60%
Agent Automation	IV, V	Converting roles to AI agents	~40%

mermaid

flowchart LR

subgraph "The Human Workflow"

P1[Part I
The Problem]

P2[Part II
Human Roles]

P3[Part III
Artifact Flow]

end

subgraph "Agent Automation"

P4[Part IV
Agent Design]

P5[Part V
Orchestration]

end

P1 --> P2 --> P3 --> P4 --> P5

style P1 fill:#ffcccb,stroke:#333

style P2 fill:#a8e6cf,stroke:#333

style P3 fill:#ffd3a5,stroke:#333

style P4 fill:#4ecdc4,stroke:#333

style P5 fill:#6bc777,stroke:#333

THE HUMAN WORKFLOW

Part I: The Planning Problem in Data Engineering

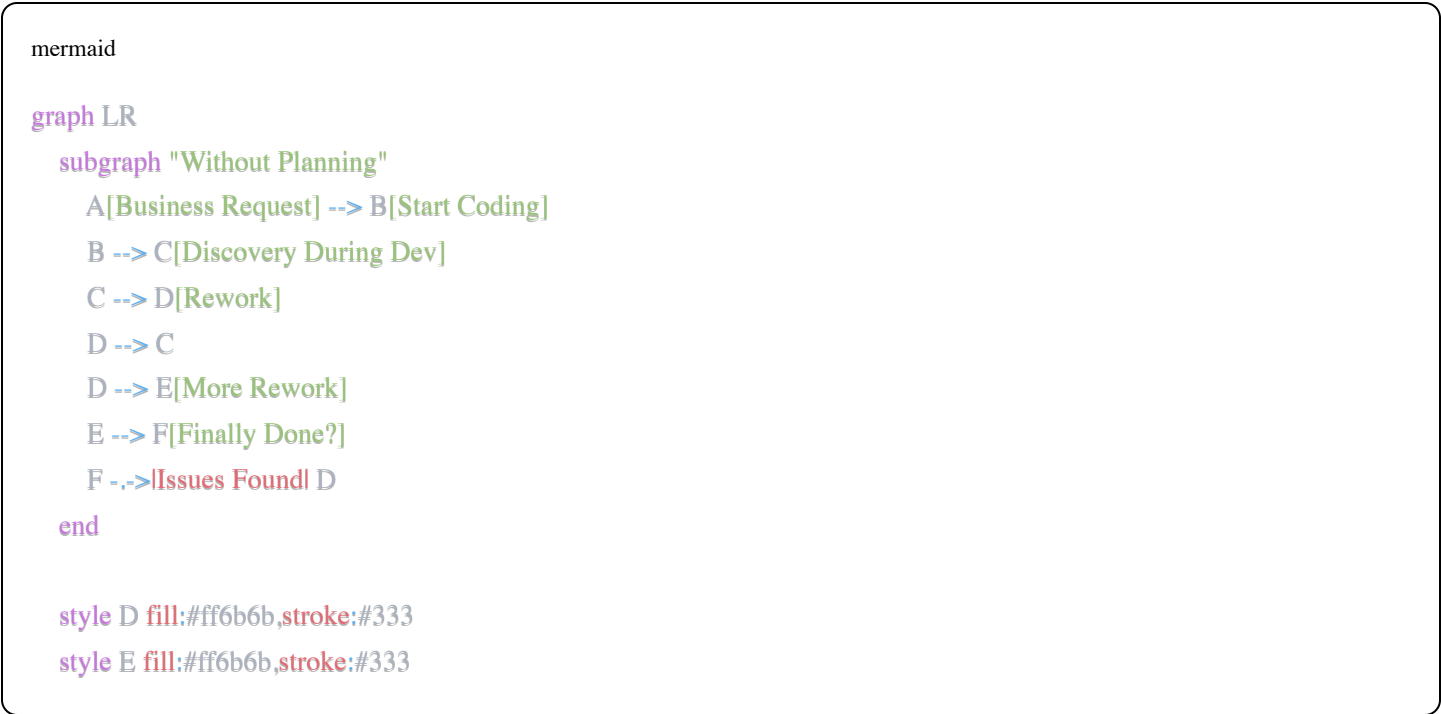
3.1 Why Data Engineering Projects Fail Without Proper Planning

3.1.1 The "Just Start Coding" Anti-Pattern

Many data engineering projects begin with enthusiasm and end in frustration. A stakeholder says "we need customer data in the warehouse," and engineers immediately start writing Spark jobs. Three months later, the pipeline exists but:

- Business rules are hardcoded and undocumented
- No one knows which source columns map to which targets
- Data quality issues surface in production
- Reconciliation fails and no one can explain why

3.1.2 The Hidden Cost of Rework



The rework multiplier in data engineering:

When Issue Discovered	Cost to Fix	Example
Requirements Phase	1x	"We need order history too"
Design Phase	3x	"This won't scale to 100M rows"
Development Phase	10x	"Source has duplicates we didn't know about"
Testing Phase	30x	"Reconciliation is off by \$2M"
Production	100x	"Finance reported wrong numbers to SEC"

3.1.3 War Story: The Missing Mapping Document

Case Study: The \$2M Reconciliation Nightmare

A retail company built a customer lifetime value pipeline without a formal Data Mapping Document. The pipeline ran for 6 months before Finance noticed LTV calculations were consistently 15% higher than their legacy system.

Root Cause: A developer interpreted "total_amount" as including tax, while the source system stored it without tax. This was never documented.

Cost:

- 3 weeks of forensic analysis
- 6 months of incorrect executive dashboards

- Restated financial projections
- Complete pipeline rebuild

Prevention: A 2-hour mapping review session would have caught this on day one.

3.1.4 Data Engineering vs. Software Engineering: Why Planning Differs

mermaid

graph TB

subgraph "Software Engineering"

SE1[User Story] --> SE2[Design]

SE2 --> SE3[Code]

SE3 --> SE4[Test]

SE4 --> SE5[Deploy]

end

subgraph "Data Engineering"

DE1[Business Request] --> DE2[Source Analysis]

DE2 --> DE3[Architecture Design]

DE3 --> DE4[Data Modeling]

DE4 --> DE5[Column Mappings]

DE5 --> DE6[Quality Rules]

DE6 --> DE7[Pipeline Code]

DE7 --> DE8[Reconciliation]

DE8 --> DE9[Deploy]

end

style DE2 fill:#4ecdc4,stroke:#333

style DE4 fill:#4ecdc4,stroke:#333

style DE5 fill:#4ecdc4,stroke:#333

style DE6 fill:#4ecdc4,stroke:#333

style DE8 fill:#4ecdc4,stroke:#333

Key differences that demand specialized planning:

Aspect	Software Engineering	Data Engineering
Primary Input	User requirements	Source system data
Transformation Logic	Business logic in code	Column-level mappings
Testing	Unit/Integration tests	Data quality + Reconciliation
Documentation	API specs	Mapping documents + Lineage
Failure Mode	App crashes	Silent data corruption
Debugging	Stack traces	Row-level forensics

3.1.5 The Silent Failure Problem

Data pipelines don't crash with stack traces—they silently produce wrong data. This makes comprehensive upfront planning not optional but essential.

The Pipeline That Never Failed

A telco's usage pipeline ran successfully for 18 months with green dashboards everywhere. Then a billing audit discovered 4% of records were being silently dropped due to a timezone edge case. Total revenue impact: \$4.2M in unbilled usage.

The pipeline never threw an error. Data quality rules would have caught it on day one.

3.2 The Artifact Chain: What Data Projects Actually Need

3.2.1 Beyond the PRD: Data-Specific Artifacts

A Product Requirements Document (PRD) tells you *what* the business wants. Data engineering requires additional artifacts that specify *how* data flows, transforms, and validates.

mermaid

graph TD

subgraph "Software Project Artifacts"

S1[PRD]

S2[Technical Design]

S3[API Specs]

S4[Test Plan]

end

subgraph "Data Engineering Artifacts"

D1[Data Requirements Document
—————
a.k.a. DRD, Data Spec]

D2[High-Level Design
—————
a.k.a. HLD, Architecture Doc]

D3[Data Model Specifications
—————
a.k.a. Schema Definitions]

D4[Data Mapping Document
—————
a.k.a. DMD, Mapping Spec, ETL S

D5[Data Quality Specification
—————
a.k.a. DQS, DQ Rules]

D6[Low-Level Design
—————
a.k.a. LLD, Technical Spec]

D7[Epics & Stories
—————
a.k.a. Tickets, Work Items]

end

D1 --> D2

D2 --> D3

D3 --> D4

D4 --> D5

D5 --> D6

D6 --> D7

style D1 fill:#a8e6cf,stroke:#333

style D2 fill:#dcedc1,stroke:#333

style D3 fill:#ffd3a5,stroke:#333

style D4 fill:#ffaaa5,stroke:#333

style D5 fill:#ff8b94,stroke:#333

style D6 fill:#d4a5ff,stroke:#333

style D7 fill:#a5d4ff,stroke:#333

Terminology Note

Organizations use different names for these artifacts. Throughout this chapter, we use:

- **DRD** (Data Requirements Document) - some call it "Data Spec" or "Requirements Spec"
- **DMD** (Data Mapping Document) - some call it "Mapping Spec", "ETL Spec", or "Source-to-Target Mapping"
- **DQS** (Data Quality Specification) - some call it "DQ Rules" or "Validation Spec"

The names matter less than the content. Use whatever terminology your organization prefers.

3.2.2 The Dependency Chain

Each artifact builds on the previous ones. Skipping an artifact creates gaps that surface during development or — worse — production.

mermaid

graph LR

```
DRD[Data Requirements<br/>Document] -->|"What data?<br/>What rules?"| HLD[High-Level<br/>Design]
HLD -->|"What architecture?<br/>What tech?"| DM[Data Model<br/>Specs]
DM -->|"What schemas?<br/>What types?"| DMD[Data Mapping<br/>Document]
DMD -->|"What transforms?<br/>What lineage?"| DQS[Data Quality<br/>Specification]
DQS -->|"What checks?<br/>What thresholds?"| LLD[Low-Level<br/>Design]
LLD -->|"What tasks?<br/>What sequence?"| Stories[Epics &<br/>Stories]
```

```
style DRD fill:#a8e6cf,stroke:#333
style HLD fill:#dcedc1,stroke:#333
style DM fill:#ffd3a5,stroke:#333
style DMD fill:#ffaaa5,stroke:#333
style DQS fill:#ff8b94,stroke:#333
style LLD fill:#d4a5ff,stroke:#333
style Stories fill:#a5d4ff,stroke:#333
```

3.2.3 Choosing Your Workflow Depth

Not every project needs every artifact at full depth. Use this guide to calibrate:

mermaid

quadrantChart

```
title Project Complexity vs Team Size
x-axis Small Team --> Large Team
y-axis Simple Project --> Complex Project
quadrant-1 Full Workflow + Dedicated Roles
quadrant-2 Full Workflow + Shared Roles
quadrant-3 Abbreviated Workflow
quadrant-4 Standard Workflow
```

```
"POC / Spike": [0.2, 0.15]
"Single Source Pipeline": [0.3, 0.35]
"Multi-Source Integration": [0.5, 0.65]
"Enterprise Data Platform": [0.8, 0.85]
"Startup MVP": [0.25, 0.45]
"Department Analytics": [0.4, 0.5]
```

Project Type	Recommended Artifacts	Depth
POC / Spike	DRD (brief), DMD (key columns only)	Light
Single Source Pipeline	DRD, HLD, DMD, DQS	Standard
Multi-Source Integration	All artifacts	Full
Enterprise Platform	All artifacts + governance addendum	Full+

3.2.4 Introduction to Planning-with-Intent

Planning-with-Intent (PWI) is a framework that:

- 1. **Defines clear roles** with specific inputs and outputs
 - 2. **Provides templates** for each artifact type
 - 3. **Establishes review gates** between phases
 - 4. **Enables automation** through AI agents while keeping humans in control
- mermaid

flowchart TB

subgraph "Planning-with-Intent Framework"

direction TB

subgraph "Phase 1: Requirements"

R1[Business Request]

R2[Data Analyst]

R3[DRD]

R1 --> R2 --> R3

end

subgraph "Phase 2: Design"

D1[Data Architect]

D2[HLD]

D3[Data Modeler]

D4[Schemas]

R3 --> D1 --> D2

D2 --> D3 --> D4

end

subgraph "Phase 3: Specification"

M1[Mapping Engineer]

M2[DMD]

Q1[DQ Engineer]

Q2[DQS]

D4 --> M1 --> M2

M2 --> Q1 --> Q2

end

subgraph "Phase 4: Implementation Planning"

T1[Technical Lead]

T2[LLD]

S1[Scrum Master]

S2[Stories]

Q2 --> T1 --> T2

T2 --> S1 --> S2

end

end

RG1{Review Gate}

RG2{Review Gate}

RG3{Review Gate}

R3 --> RG1 --> D1

D4 --> RG2 --> M1

Q2 --> RG3 --> T1

style RG1 fill:#ffd700,stroke:#333

style RG2 fill:#ffd700,stroke:#333

style RG3 fill:#ffd700,stroke:#333

Part II: The Human Roles in Data Engineering Planning

3.3 Role Overview: The Planning Team

3.3.1 The Six Core Functions

Data engineering planning involves six distinct functions. These may be performed by six specialists or by two people wearing multiple hats—the functions remain the same.

mermaid

graph TB

subgraph "The Six Planning Functions"

F1[1. Business/Data Analyst
Requirements & Rules]

F2[2. Data Architect
Architecture & Tech Stack]

F3[3. Data Modeler
Schema Design]

F4[4. Mapping Engineer
Transformations & Lineage]

F5[5. DQ Engineer
Quality & Reconciliation]

F6[6. Scrum Master
Stories & Delivery]

end

TL[Technical Lead
LLD & Implementation]

F1 --> F2 --> F3 --> F4 --> F5 --> TL --> F6

style F1 fill:#a8e6cf,stroke:#333

style F2 fill:#dcedc1,stroke:#333

style F3 fill:#ffd3a5,stroke:#333

style F4 fill:#ffaaa5,stroke:#333

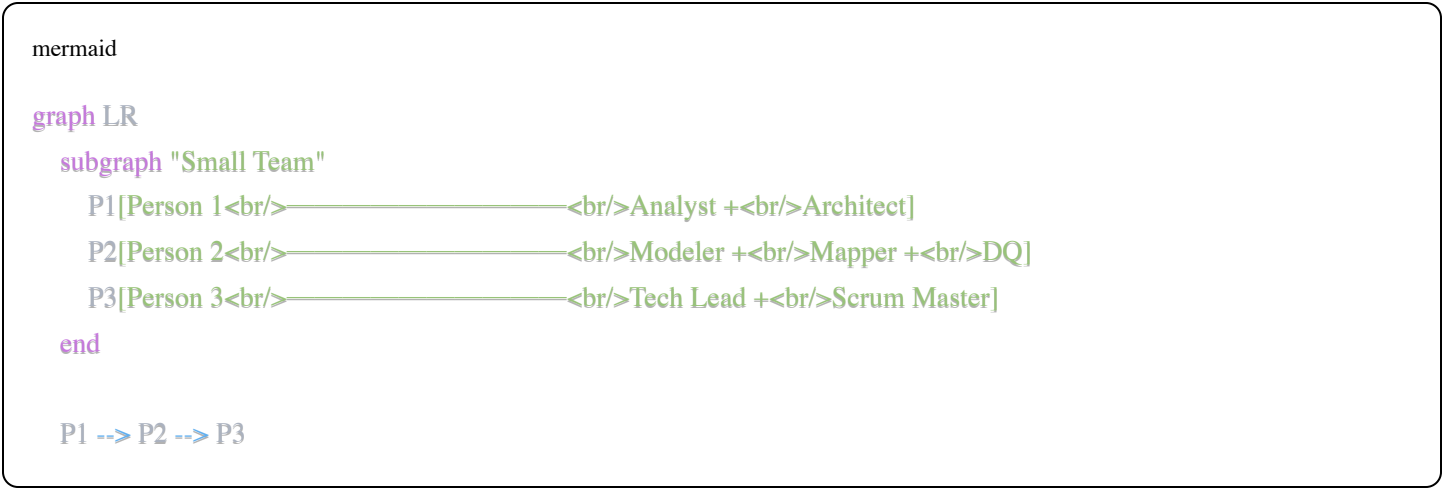
style F5 fill:#ff8b94,stroke:#333

style TL fill:#d4a5ff,stroke:#333

style F6 fill:#a5d4ff,stroke:#333

3.3.2 Organizational Patterns

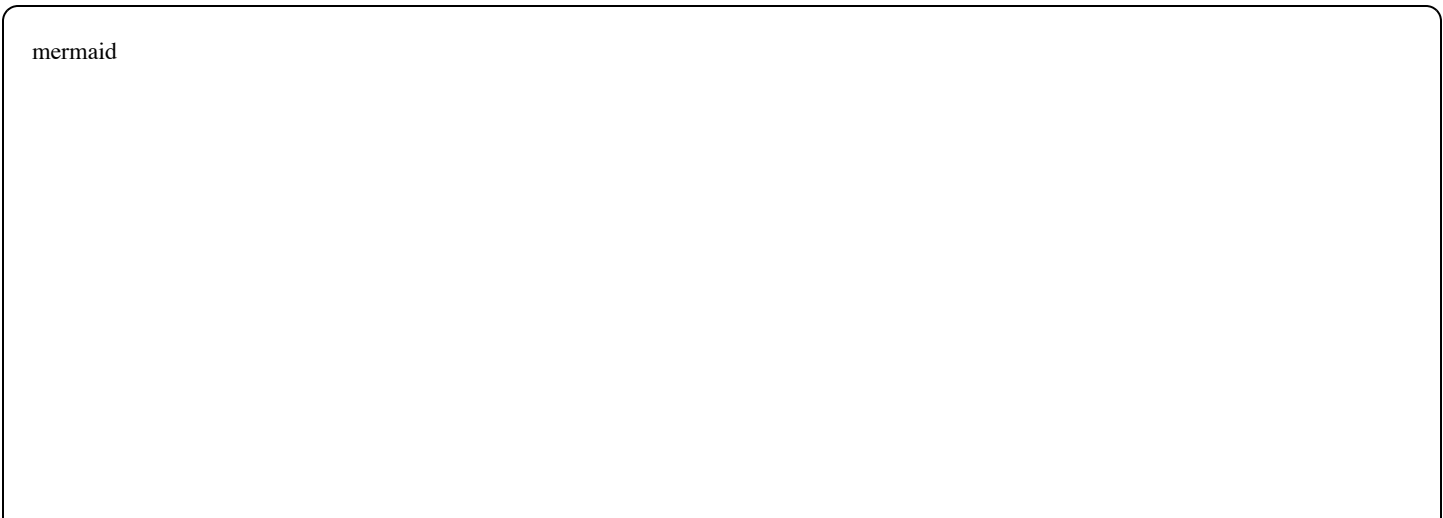
Pattern A: Small Team (2-3 people)



Pattern B: Enterprise Team (Specialists)



Pattern C: Solo Practitioner + Agents



graph LR

subgraph "Solo + AI Agents"

HUMAN[Data Engineer
—————
Reviews &
Decisions]

A1[Agent: Analyst]

A2[Agent: Architect]

A3[Agent: Modeler]

A4[Agent: Mapper]

A5[Agent: DQ]

A6[Agent: Stories]

end

A1 -->|Review| HUMAN

HUMAN -->|Approve| A2

A2 -->|Review| HUMAN

HUMAN -->|Approve| A3

A3 -->|Review| HUMAN

HUMAN -->|Approve| A4

A4 -->|Review| HUMAN

HUMAN -->|Approve| A5

A5 -->|Review| HUMAN

HUMAN -->|Approve| A6

style HUMAN fill:#ffd700,stroke:#333,stroke-width:3px

🔑 Key Insight: Agents Help Small Teams Punch Above Their Weight

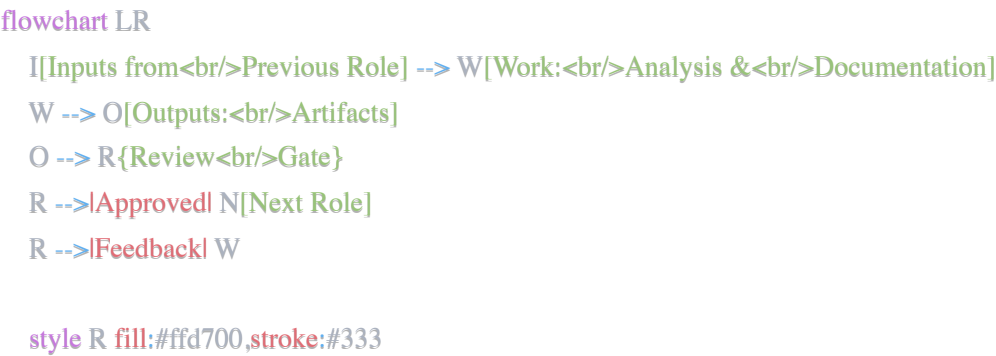
A solo data engineer with AI agents can produce the same quality artifacts as a 5-person team—if they understand what each role does and review agent outputs carefully.

The agents generate comprehensive first drafts. The human provides judgment, context, and approval.

3.3.3 The Handoff Pattern

Every role follows the same pattern:

mermaid



3.3.4 Review Gates for Small Teams

When you're reviewing your own work (or an agent's output), use this checklist:

Gate	Key Questions
Post-DRD	Are all sources identified? Business rules complete? SLAs specific?
Post-HLD	Is the architecture justified? Tech stack appropriate? Scalable?
Post-Schema	Do schemas follow standards? Keys defined? Types correct?
Post-DMD	Every target column mapped? Null handling specified? Lineage complete?
Post-DQS	Critical rules identified? Thresholds realistic? Actions defined?
Post-LLD	Tasks clear? Dependencies mapped? Error handling defined?

3.4 The Business/Data Analyst

3.4.1 Why This Role Exists

The Business/Data Analyst bridges the gap between stakeholders who know what they need and engineers who will build it. Without this role, requirements remain ambiguous, leading to rework.

3.4.2 Key Responsibilities

mermaid

mindmap

root((Business/Data
Analyst))

Source Discovery

Identify source systems

Document access methods

Catalog available tables

Estimate data volumes

Business Rules

Elicit calculations

Document derivations

Capture edge cases

Define defaults

Consumer Requirements

Identify data consumers

Define SLAs

Specify freshness needs

Document access patterns

Data Quality Expectations

Critical fields

Valid value ranges

Referential requirements

Tolerance thresholds

3.4.3 Inputs and Outputs

mermaid

flowchart LR

subgraph Inputs

I1[Business Request]

I2[Stakeholder Interviews]

I3[Source System Docs]

I4[Existing Data Catalog]

end

subgraph "Analyst Work"

A[Business/Data
Analyst]

end

subgraph Outputs

O1[Data Requirements
Document - DRD]

end

I1 --> A

I2 --> A

I3 --> A

I4 --> A

A --> O1

style O1 fill:#a8e6cf,stroke:#333,stroke-width:3px

3.4.4 Data Requirements Document (DRD) — Abbreviated Template

markdown

Data Requirements Document: [Project Name]

1. Executive Summary

- **Business Objective:** [One sentence]
- **Data Products:** [List of outputs]
- **Success Metrics:** [How we measure success]

2. Source Systems

System	Type	Tables	Est. Volume	Access Method	CDC?
CRM	PostgreSQL	customers, orders	10M rows	JDBC	Yes

3. Target Requirements

Data Product	Layer	Grain	Consumers	SLA
dim_customer	Gold	Customer	BI, ML	T+1 6AM

3. Business Rules

Rule ID	Name	Logic
BR001	LTV Calculation	<code>SUM(order_total) * 0.3 * 5</code>
BR002	Active Customer	<code>last_order_date >= TODAY - 365</code>

5. Data Quality Requirements

Rule ID	Entity	Rule	Threshold	Action
DQ001	customer	customer_id NOT NULL	100%	Reject
DQ002	customer	email valid format	99%	Flag

6. Governance

- **PII Columns:** email, phone, address
- **Retention:** Gold: 7 years, Bronze: 90 days
- **Compliance:** GDPR Article 17 (right to erasure)

7. Open Questions

- [] Is SSN required in the output?
- [] What timezone for order timestamps?

Full template available in Appendix A.1

3.4.5 Skills Needed

- Domain knowledge of the business area

- Requirements elicitation techniques
- SQL proficiency for source exploration
- Documentation and communication

3.4.6 Common Pitfalls and Prevention

Pitfall	Consequence	Prevention
Incomplete source discovery	Missing data in pipeline	Systematic catalog review
Undocumented business rules	Hardcoded logic, no traceability	Rule template enforcement
Vague SLAs ("near real-time")	Misaligned expectations	Specific targets: "< 15 min latency"
Missing PII identification	Compliance violations	PII checklist review
Assumed data quality	Production surprises	Profile source data before sign-off



Pro Tip: Profile Before You Commit

Run basic profiling queries on source data before finalizing the DRD:

```
sql

-- Quick source profiling
SELECT
  COUNT(*) as row_count,
  COUNT(DISTINCT customer_id) as unique_customers,
  SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END) as null_emails,
  MIN(created_at) as earliest_record,
  MAX(created_at) as latest_record
FROM source.customers;
```

This takes 5 minutes and prevents weeks of rework.

3.5 The Data Architect

3.5.1 Why This Role Exists

The Data Architect translates requirements into a technical blueprint. They make foundational decisions about architecture patterns, technology choices, and system design that guide all subsequent work.

3.5.2 Key Responsibilities

mindmap

root((Data
Architect))

Architecture Pattern

Medallion Bronze/Silver/Gold

Lambda real-time + batch

Kappa streaming-first

Data Vault auditability

Layer Design

Landing zone

Bronze raw

Silver cleansed

Gold business-ready

Technology Selection

Orchestration

Processing engine

Storage format

Serving layer

Scalability Planning

Current volumes

Growth projections

Performance targets

CDC Strategy

Debezium/Kafka

Timestamp-based

Full refresh fallback

3.5.3 Inputs and Outputs

mermaid

flowchart LR

subgraph Inputs

I1[Data Requirements
Document - DRD]

I2[Technology Catalog]

I3[Infrastructure Constraints]

I4[Team Capabilities]

end

subgraph "Architect Work"

A[Data
Architect]

end

subgraph Outputs

O1[High-Level Design
HLD / Architecture Doc]

end

I1 --> A

I2 --> A

I3 --> A

I4 --> A

A --> O1

style I1 fill:#a8e6cf,stroke:#333

style O1 fill:#dcedc1,stroke:#333,stroke-width:3px

3.5.4 Architecture Pattern Decision Framework

mermaid

flowchart TD

Q1{Real-time
Required?}

Q1 -->|No| Q2{Audit/Compliance
Critical?}

Q1 -->|Yes| Q3{Batch Also
Needed?}

Q2 -->|No| M[Medallion
Architecture]

Q2 -->|Yes| DV[Data Vault
Architecture]

Q3 -->|No| K[Kappa
Architecture]

Q3 -->|Yes| L[Lambda
Architecture]

style M fill:#4ecdc4,stroke:#333

style DV fill:#ff6b6b,stroke:#333

style K fill:#ffd93d,stroke:#333

style L fill:#6bcb77,stroke:#333

3.5.5 CDC and Change Capture Considerations

When designing for CDC sources, the HLD must document:

mermaid

flowchart TD

subgraph "CDC Decision Tree"

Q1{CDC Available
in Source?}

Q1 -->|Yes| Q2{Debezium
Compatible?}

Q1 -->|No| TS[Timestamp-Based
Incremental]

Q2 -->|Yes| DEB[Debezium + Kafka]

Q2 -->|No| Q3{Other CDC
Tool?}

Q3 -->|Yes| OTHER[Vendor CDC Tool]

Q3 -->|No| TS

TS --> FULL[Full Refresh
Fallback Plan]

end

style DEB fill:#4ecdc4,stroke:#333

style TS fill:#ffd93d,stroke:#333

style OTHER fill:#6bcb77,stroke:#333

CDC Architecture Questions to Answer in HLD:

Question	Document In HLD
What's the replication lag SLA?	"CDC lag must be < 5 minutes"
How are deletes captured?	"Tombstone records with _deleted flag"
Schema evolution strategy?	"Schema registry with backward compatibility"
Slot/publication management?	"Dedicated slot per pipeline, monitored for lag"
Fallback if CDC fails?	"Automatic switch to timestamp-based with alert"

3.5.6 Capacity Planning Section

The HLD should include resource and cost projections:

markdown

Capacity Planning

Current State

| Metric | Value |

|-----|-----|

| Daily Source Volume | 50M rows |

| Storage (Bronze) | 500 GB |

| Processing Time | 45 minutes |

Year 1 Projection

| Metric | Value | Growth |

|-----|-----|-----|

| Daily Source Volume | 200M rows | 4x |

| Storage (Total) | 3 TB | 4x |

Scaling Triggers

- If daily volume > 500M rows → Upgrade cluster

- If processing time > 3 hours → Add parallelism

3.5.7 High-Level Design (HLD) — Abbreviated Template

markdown

```
# High-Level Design: [Pipeline Name]

## 1. Architecture Overview
- Pattern: Medallion (Bronze → Silver → Gold)
- Justification: Batch workload, no real-time requirement

## 2. Layer Specifications
| Layer | Purpose | Retention | Quality Gates |
|-----|-----|-----|-----|
| Bronze | Raw preservation | 90 days | Schema validation |
| Silver | Cleansed, canonical | 2 years | Business rules |
| Gold | Business-ready | 7 years | Full DQ + Recon |

## 3. Technology Stack
| Component | Selection | Rationale |
|-----|-----|-----|
| Orchestration | [Tool] | [Why] |
| Processing | [Tool] | [Why] |
| Storage | [Format] | [Why] |
| Quality | [Framework] | [Why] |

## 3. Integration Points
| System | Direction | Method | Frequency |
|-----|-----|-----|-----|
| CRM | Inbound | CDC | Real-time |
| Warehouse | Outbound | Batch | Daily |

## 5. Capacity Planning
[See 4.5.6 above]

## 6. Security & DR
- Encryption: At-rest and in-transit
- RTO/RPO: 4 hours / 1 hour
```

Full template available in Appendix A.2

3.5.8 Medallion Architecture Reference

mermaid

graph LR

subgraph Sources

S1[(OLTP)]

S2[(Files)]

S3[(APIs)]

end

subgraph "Bronze Layer"

B1[Raw Tables
—————
Source-aligned
Append-only
Partitioned by date]

end

subgraph "Silver Layer"

SL1[Clean Tables
—————
Deduplicated
Standardized
Type-enforced]

end

subgraph "Gold Layer"

G1[Dimensions]

G2[Facts]

G3[Aggregates]

end

subgraph Consumers

C1[BI Tools]

C2[ML Platform]

C3[APIs]

end

S1 --> B1

S2 --> B1

S3 --> B1

B1 --> SL1

SL1 --> G1

SL1 --> G2

G1 --> G3

G2 --> G3

G1 --> C1

G2 --> C1

G3 --> C1

G1 --> C2

G2 --> C2

G1 --> C3

style B1 fill:#cd7f32,stroke:#333

style SL1 fill:#c0c0c0,stroke:#333

style G1 fill:#ffd700,stroke:#333

style G2 fill:#ffd700,stroke:#333

style G3 fill:#ffd700,stroke:#333

3.5.9 Note: Data Modeling Responsibilities

In some organizations, the Data Architect also handles data modeling. In others, a dedicated Data Modeler takes over after the HLD is complete. The next section covers the Data Modeler role for organizations that separate these functions.

When to Skip Section 3.6 (Data Modeler)

If your team is small (< 5 people) or your project has simple schemas, the Architect typically handles modeling as part of the HLD. Skip to Section 3.7 (Mapping Engineer) and incorporate schema definitions into your HLD.

3.6 The Data Modeler (Optional Dedicated Role)

3.6.1 Why This Role Exists

The Data Modeler designs the schema structures at each layer of the architecture. While the Architect defines *what layers exist*, the Modeler specifies *what tables and columns exist in each layer*.

3.6.2 When to Have a Dedicated Modeler

Scenario	Recommendation
Small project, simple schemas	Architect handles modeling
Enterprise with complex domains	Dedicated Modeler
Multiple subject areas	Dedicated Modeler
Strict naming/governance standards	Dedicated Modeler
Team < 5 people	Architect handles modeling

3.6.3 Key Responsibilities

mermaid

mindmap

root((Data
Modeler))

Bronze Schemas

Source-aligned tables

Metadata columns

Partition strategy

Silver Schemas

Canonical entities

Cleansed structures

Type standardization

Gold Schemas

Dimensional model

Fact tables

Aggregate tables

Standards

Naming conventions

Data types

Nullability rules

Documentation

3.6.4 Inputs and Outputs

mermaid

flowchart LR

subgraph Inputs

I1[Data Requirements
Document - DRD]
I2[High-Level Design
HLD]
I3[Enterprise Data
Standards]

end

subgraph "Modeler Work"

A[Data
Modeler]

end

subgraph Outputs

O1[Bronze Schema
Definitions]
O2[Silver Schema
Definitions]
O3[Gold Schema
Definitions]

end

I1 --> A

I2 --> A

I3 --> A

A --> O1

A --> O2

A --> O3

style I1 fill:#a8e6cf,stroke:#333

style I2 fill:#dcedc1,stroke:#333

style O1 fill:#cd7f32,stroke:#333,stroke-width:2px

style O2 fill:#c0c0c0,stroke:#333,stroke-width:2px

style O3 fill:#ffd700,stroke:#333,stroke-width:2px

3.6.5 SCD Type Decision Framework

mermaid

flowchart TD

Q1{Need to track
historical changes?}

Q1 -->|No| SCD1[SCD Type 1
—————
Overwrite]

Q1 -->|Yes| Q2{Need full
history?}

Q2 -->|No| SCD3[SCD Type 3
—————
Previous + Current]

Q2 -->|Yes| SCD2[SCD Type 2
—————
Version rows with
effective dates]

style SCD1 fill:#4ecdc4,stroke:#333

style SCD2 fill:#ff6b6b,stroke:#333

style SCD3 fill:#ffd93d,stroke:#333

3.7 The Mapping Analyst/Engineer

3.7.1 Why This Role Exists

The Mapping Engineer creates the most detail-intensive artifact in data engineering: the Data Mapping Document (DMD). This document specifies exactly how each source column transforms into each target column across all pipeline stages.

3.7.2 Key Responsibilities

mermaid

mindmap

root((Mapping
Engineer))

Column Mappings

Source to Bronze

Bronze to Silver

Silver to Gold

Transformation Logic

Type conversions

Calculations

Aggregations

Lookups

Null Handling

Default values

Coalesce rules

Rejection criteria

Lineage Documentation

End-to-end tracking

Business rule references

Change detection

3.7.3 Inputs and Outputs

mermaid

flowchart LR

subgraph Inputs

I1[Data Requirements
Document - DRD]

I2[High-Level Design
HLD]

I3[Data Model
Specifications]

I4[Source System
Metadata]

end

subgraph "Mapping Work"

A[Mapping
Engineer]

end

subgraph Outputs

O1[Data Mapping
Document - DMD]

end

I1 --> A

I2 --> A

I3 --> A

I4 --> A

A --> O1

style I1 fill:#a8e6cf,stroke:#333

style I2 fill:#dcedc1,stroke:#333

style I3 fill:#ffd3a5,stroke:#333

style O1 fill:#ffaaa5,stroke:#333,stroke-width:3px

3.7.4 Data Mapping Document (DMD) — Abbreviated Template

markdown

Data Mapping Document: [Pipeline Name]

1. Pipeline Overview

| Attribute | Value |

|-----|-----|

| Pipeline ID | PL-CUST-001 |

| Source | CRM (PostgreSQL) |

| Target | Data Warehouse |

| Pattern | Incremental (CDC) |

| Schedule | Daily 02:00 UTC |

2. Stage Mappings

2.1 Source → Bronze

| # | Source Column | Type | Target Column | Type | Transform | Null Handling |

|---|-----|-----|-----|-----|-----|-----|

| 1 | customer_id | VARCHAR | customer_id | STRING | CAST | Reject |

| 2 | email | VARCHAR | email | STRING | LOWER | Allow |

| 3 | - | - | _ingestion_ts | TIMESTAMP | NOW() | System |

2.2 Bronze → Silver

| # | Source | Target | Transform | Business Rule |

|---|-----|-----|-----|-----|

| 1 | first_name, last_name | full_name | CONCAT | BR-001 |

| 2 | email | email_domain | SPLIT('@')[1] | - |

2.3 Silver → Gold

| # | Source | Target | Transform | Business Rule |

|---|-----|-----|-----|-----|

| 1 | - | customer_key | SURROGATE | - |

| 2 | - | lifetime_value | CALCULATED | BR-002 |

| 3 | - | segment | CASE WHEN | BR-003 |

3. Business Rules

| ID | Name | Logic |

|----|-----|-----|

| BR-001 | Full Name | `CONCAT(first\_name, ' ', last\_name)` |

| BR-002 | LTV | `SUM(order\_total) \* 0.3 \* 5` |

| BR-003 | Segment | `CASE WHEN LTV >= 10000 THEN 'Platinum'...` |

3. Edge Cases

| Scenario | Handling |

|-----|-----|

Null in customer_id	Reject record, alert
Duplicate keys	Take latest by updated_at
Late-arriving data	72-hour merge window

Full template available in Appendix A.4

3.7.5 Handling Common Edge Cases

Document explicit handling for these scenarios in the DMD:

Scenario	Standard Approach	Example
Source column removed	Flag as breaking change, alert	customer.middle_name dropped
New source column	Add to Bronze, assess for Silver/Gold	New customer.loyalty_tier
Type change in source	Bronze: accept as string, Silver: handle conversion	age changed from INT to STRING
Null in required field	Quarantine record + alert	order.customer_id IS NULL
Duplicate keys	Take latest by timestamp, log duplicates	Multiple records for same order_id
Late-arriving data	Merge window or reprocessing strategy	Order arrives 3 days late
Invalid foreign key	Quarantine or assign to "Unknown" dimension	order.customer_id not in customers
Timezone ambiguity	Document assumed timezone, convert to UTC	created_at has no timezone

3.7.6 End-to-End Lineage Diagram

mermaid

flowchart LR

subgraph "Source"

S1[crm.customers
]—————
customer_id
first_name
last_name
email
create

end

subgraph "Bronze"

B1[raw_customers
]—————
customer_id
first_name
last_name
email
creat

end

subgraph "Silver"

SL1[clean_customer
]—————
customer_id
full_name
email
email_domain

end

subgraph "Gold"

G1[dim_customer
]—————
customer_key
customer_id
full_name
email_maske

end

S1 -->|"CAST
Add metadata"| B1

B1 -->|"CONCAT names
Extract domain
Validate email"| SL1

SL1 -->|"Surrogate key
Mask PII
Calculate LTV
SCD Type 2"| G1

style S1 fill:#e8e8e8,stroke:#333

style B1 fill:#cd7f32,stroke:#333

style SL1 fill:#c0c0c0,stroke:#333

style G1 fill:#ffd700,stroke:#333

3.8 The Data Quality Engineer

3.8.1 Why This Role Exists

The DQ Engineer defines the rules that ensure data is correct, complete, and consistent. Without explicit quality specifications, issues surface in production dashboards rather than pipeline logs.

3.8.2 Key Responsibilities

mermaid

mindmap

root((DQ
Engineer))

Quality Rules

Completeness NOT NULL

Uniqueness no duplicates

Validity format/range

Consistency cross-field

Reconciliation

Row counts

Sum checks

Key matching

Anomaly Detection

Volume changes

Distribution shifts

Null rate changes

Actions

Reject

Quarantine

Flag

Alert

3.8.3 Inputs and Outputs

mermaid

flowchart LR

subgraph Inputs

I1[Data Requirements
Document - DRD]

I2[Data Mapping
Document - DMD]

I3[Sample Source
Data]

end

subgraph "DQ Work"

A[DQ
Engineer]

end

subgraph Outputs

O1[Data Quality
Specification - DQS]

O2[Framework Config
- tool-specific]

end

I1 --> A

I2 --> A

I3 --> A

A --> O1

A --> O2

style I1 fill:#a8e6cf,stroke:#333

style I2 fill:#ffaaa5,stroke:#333

style O1 fill:#ff8b94,stroke:#333,stroke-width:3px

3.8.4 Data Quality Specification (DQS) — Abbreviated Template

markdown

```
# Data Quality Specification: [Pipeline Name]

## 1. Overview
- Pipeline: Customer Dimension
- Framework: [Your DQ tool]
- Alert Channel: #data-alerts

## 2. Rules by Layer

### Bronze Rules
| Rule ID | Column | Check | Threshold | Severity | Action |
|-----|-----|-----|-----|-----|-----|
| BRZ-001 | customer_id | NOT NULL | 100% | Critical | Reject |
| BRZ-002 | _ingestion_ts | NOT NULL | 100% | Critical | Reject |

### Silver Rules
| Rule ID | Column | Check | Threshold | Severity | Action |
|-----|-----|-----|-----|-----|-----|
| SLV-001 | customer_id | UNIQUE | 100% | Critical | Reject |
| SLV-002 | email | REGEX | 99% | Warning | Flag |

### Gold Rules
| Rule ID | Column | Check | Threshold | Severity | Action |
|-----|-----|-----|-----|-----|-----|
| GLD-001 | customer_key | UNIQUE | 100% | Critical | Halt |
| GLD-002 | lifetime_value | >= 0 | 100% | Critical | Halt |

## 3. Reconciliation
| Metric | Source Query | Target Query | Tolerance |
|-----|-----|-----|-----|
| Row Count | `COUNT(*) FROM src` | `COUNT(*) FROM tgt` | 0.01% |
| Sum Amount | `SUM(amount) FROM src` | `SUM(amount) FROM tgt` | $100 |

## 3. Freshness
| Table | Column | Max Staleness | Check Frequency |
|-----|-----|-----|-----|
| dim_customer | _loaded_at | 25 hours | Hourly |
```

Full template available in Appendix A.5

3.8.5 Quality Rule Severity Framework

flowchart TD

subgraph "Rule Evaluation"

R1[Execute DQ Rule]

R1 --> R2{Pass?}

end

R2 -->|Yes| PASS[Continue Pipeline]

R2 -->|No| SEV{Severity?}

SEV -->|Critical| C1[Halt Pipeline]

SEV -->|Warning| W1[Alert Team]

SEV -->|Info| I1[Log Only]

C1 --> C2[Create Incident]

W1 --> W2[Continue Pipeline]

I1 --> I2[Continue Pipeline]

style PASS fill:#4ecdc4,stroke:#333

style C1 fill:#ff6b6b,stroke:#333

style W1 fill:#ffd93d,stroke:#333

style I1 fill:#a8e8e8,stroke:#333

3.9 The Technical Lead / Senior Engineer

3.9.1 Why This Role Exists

The Technical Lead transforms specifications into an executable implementation plan. They design the DAG structure, define error handling strategies, and create the Low-Level Design that guides developers.

3.9.2 Key Responsibilities

mermaid

mindmap

root((Technical
Lead))

DAG Design

Task structure

Dependencies

Parallelization

Scheduling

Error Handling

Retry strategies

Dead letter queues

Alerting

Recovery procedures

Performance

Partitioning

Caching

Resource allocation

Implementation Planning

Task breakdown

Sequencing

Risk mitigation

3.9.3 Inputs and Outputs

mermaid

flowchart LR

subgraph Inputs

I1[HLD / Architecture]

I2[Data Model Specs]

I3[Data Mapping Doc]

I4[DQ Specification]

end

subgraph "Tech Lead Work"

A[Technical
Lead]

end

subgraph Outputs

O1[Low-Level Design
LLD]

O2[DAG Definitions]

O3[Implementation
Sequence]

end

I1 --> A

I2 --> A

I3 --> A

I4 --> A

A --> O1

A --> O2

A --> O3

style I1 fill:#dcedc1,stroke:#333

style I2 fill:#ffd3a5,stroke:#333

style I3 fill:#ffaaa5,stroke:#333

style I4 fill:#ff8b94,stroke:#333

style O1 fill:#d4a5ff,stroke:#333,stroke-width:3px

3.9.4 DAG Design Pattern

mermaid

flowchart TD

subgraph "Master DAG: customer_dimension_daily"

START[Start] --> EXTRACT

subgraph "Extract Phase"

EXTRACT[extract_customers]

end

subgraph "Bronze Phase"

BRONZE[load_bronze]

BRONZE_DQ[bronze_dq_check]

EXTRACT --> BRONZE --> BRONZE_DQ

end

subgraph "Silver Phase"

SILVER[transform_silver]

SILVER_DQ[silver_dq_check]

BRONZE_DQ --> SILVER --> SILVER_DQ

end

subgraph "Gold Phase"

GOLD[build_dimension]

GOLD_DQ[gold_dq_check]

RECON[reconciliation]

SILVER_DQ --> GOLD --> GOLD_DQ --> RECON

end

subgraph "Publish Phase"

PUBLISH[publish_to_warehouse]

CATALOG[update_catalog]

RECON --> PUBLISH --> CATALOG

end

CATALOG --> END[End]

end

style BRONZE_DQ fill:#ff8b94,stroke:#333

style SILVER_DQ fill:#ff8b94,stroke:#333

style GOLD_DQ fill:#ff8b94,stroke:#333

style RECON fill:#ff8b94,stroke:#333

3.9.5 Low-Level Design (LLD) Structure

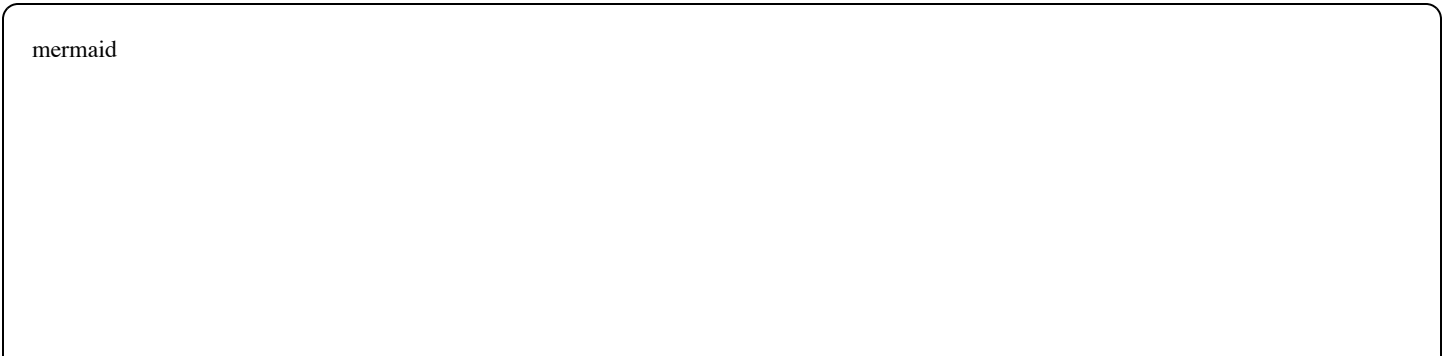


3.10 The Scrum Master / Delivery Lead

3.10.1 Why This Role Exists

The Scrum Master translates the technical plan into manageable work items. They create Epics and Stories that can be estimated, assigned, and tracked through sprint cycles.

3.10.2 Key Responsibilities



mindmap

root((Scrum
Master))

Epic Creation

Scope definition

Business value

Dependencies

Acceptance criteria

Story Decomposition

Technical tasks

Estimation

Sequencing

Assignments

Delivery Planning

Sprint allocation

Risk identification

Blocker tracking

Artifact Linking

Reference to DMD

Reference to DQS

Reference to LLD

3.10.3 Inputs and Outputs

mermaid

flowchart LR

subgraph Inputs

I1[Low-Level Design
LLD]

I2[All Previous
Artifacts]

I3[Team Capacity]

end

subgraph "Scrum Master Work"

A[Scrum
Master]

end

subgraph Outputs

O1[Epics]

O2[Stories]

O3[GitHub Issues /
Jira Tickets]

end

I1 --> A

I2 --> A

I3 --> A

A --> O1

A --> O2

O1 --> O3

O2 --> O3

style I1 fill:#d4a5ff,stroke:#333

style O1 fill:#a5d4ff,stroke:#333,stroke-width:2px

style O2 fill:#a5d4ff,stroke:#333,stroke-width:2px

style O3 fill:#a5d4ff,stroke:#333,stroke-width:3px

3.10.4 INVEST Criteria for Data Engineering Stories

Each story should be:

Criterion	Meaning	DE Example
Independent	Minimal dependencies on other stories	"Bronze schema" before "Bronze load"
Negotiable	Not over-specified	Specify <i>what</i> , not <i>how</i>
Valuable	Delivers an increment	"Load works for 1 day" is valuable
Estimable	Clear enough to size	Column mappings defined
Small	Fits in a sprint	One layer transition per story
Testable	Clear acceptance criteria	DQ rules pass

3.10.5 Epic/Story Hierarchy

mermaid

flowchart TD

subgraph "Epic: Customer Dimension Pipeline"

E1[Epic 1: Customer
Dimension Pipeline]

subgraph "Stories"

S1[Story 1.1
Bronze Schema Setup]

S2[Story 1.2
Source → Bronze Load]

S3[Story 1.3
Bronze DQ Rules]

S4[Story 1.4
Bronze → Silver Transform]

S5[Story 1.5
Silver DQ Rules]

S6[Story 1.6
Silver → Gold Dimension]

S7[Story 1.7
Gold DQ + Reconciliation]

S8[Story 1.8
DAG Integration]

end

E1 --> S1

E1 --> S2

E1 --> S3

E1 --> S4

E1 --> S5

E1 --> S6

E1 --> S7

E1 --> S8

S1 --> S2

S2 --> S3

S3 --> S4

S4 --> S5

S5 --> S6

S6 --> S7

S7 --> S8

end

style E1 fill:#a5d4ff,stroke:#333,stroke-width:3px

style S1 fill:#e8e8e8,stroke:#333

style S2 fill:#cd7f32,stroke:#333

style S3 fill:#ff8b94,stroke:#333

style S4 fill:#c0c0c0,stroke:#333

style S5 fill:#ff8b94,stroke:#333

style S6 fill:#ffd700,stroke:#333

style S7 fill:#ff8b94,stroke:#333

style S8 fill:#d4a5ff,stroke:#333

Part III: The Artifact Flow

3.11 How Artifacts Connect: The Dependency Graph

3.11.1 Complete Artifact Flow

mermaid

flowchart TD

BR[Business
Request] --> DRD[Data Requirements
Document]

DRD --> HLD[High-Level Design /
Architecture]

HLD --> DM[Data Model
Specifications]

DRD --> DMD[Data Mapping
Document]

DM --> DMD

HLD --> DMD

DRD --> DQS[Data Quality
Specification]

DMD --> DQS

HLD --> LLD[Low-Level
Design]

DM --> LLD

DMD --> LLD

DQS --> LLD

LLD --> STORIES[Epics &
Stories]

DRD --> STORIES

DMD --> STORIES

DQS --> STORIES

subgraph "Review Gates"

RG1{Gate 1}

RG2{Gate 2}

RG3{Gate 3}

RG4{Gate 4}

end

DRD --> RG1

RG1 --> HLD

DM --> RG2

RG2 --> DMD

DQS --> RG3

RG3 --> LLD

LLD --> RG4

RG4 --> STORIES

```
style BR fill:#e8e8e8,stroke:#333
style DRD fill:#a8e6cf,stroke:#333
style HLD fill:#dcedc1,stroke:#333
style DM fill:#ffd3a5,stroke:#333
style DMD fill:#ffaaa5,stroke:#333
style DQS fill:#ff8b94,stroke:#333
style LLD fill:#d4a5ff,stroke:#333
style STORIES fill:#a5d4ff,stroke:#333
style RG1 fill:#ffd700,stroke:#333
style RG2 fill:#ffd700,stroke:#333
style RG3 fill:#ffd700,stroke:#333
style RG4 fill:#ffd700,stroke:#333
```

3.11.2 Why Order Matters

If You Skip...	You Risk...
DRD	Building the wrong thing
HLD	Architecture that doesn't scale
Data Models	Inconsistent schemas across layers
DMD	Undocumented transformations
DQS	Silent data corruption
LLD	Implementation chaos

3.11.3 Review Gate Checklists

Gate	Key Questions
Gate 1: Post-DRD	All sources identified? Business rules complete? SLAs specific and measurable? PII documented?
Gate 2: Post-Schema	Schemas follow standards? Keys defined correctly? Types appropriate? Naming consistent?
Gate 3: Post-DQS	Every target column has mapping? Null handling specified? DQ rules cover critical paths? Reconciliation defined?
Gate 4: Post-LLD	Tasks clear and sized? Dependencies mapped? Error handling defined? Rollback possible?

3.12 Warning Signs Your Planning Is Failing

3.12.1 Red Flags by Phase

mermaid

flowchart TD

subgraph "Requirements Red Flags"

R1[▶ "We'll figure out
the sources later"]

R2[▶ "The business rules
are obvious"]

R3[▶ "SLA? Just make
it fast"]

end

subgraph "Design Red Flags"

D1[▶ "We don't need
a diagram"]

D2[▶ "Tech stack?
Whatever works"]

D3[▶ "CDC is too
complicated"]

end

subgraph "Mapping Red Flags"

M1[▶ "Column names are
self-explanatory"]

M2[▶ "Nulls? We'll
handle them in code"]

M3[▶ "Lineage is
overkill"]

end

subgraph "Quality Red Flags"

Q1[▶ "Source data
is clean"]

Q2[▶ "Reconciliation
takes too long"]

Q3[▶ "DQ rules slow
us down"]

end

style R1 fill:#ffcccb,stroke:#333

style R2 fill:#ffcccb,stroke:#333

style R3 fill:#ffcccb,stroke:#333

style D1 fill:#ffcccb,stroke:#333

style D2 fill:#ffcccb,stroke:#333

style D3 fill:#ffcccb,stroke:#333

style M1 fill:#ffcccb,stroke:#333

style M2 fill:#ffcccb,stroke:#333

style M3 fill:#ffcccb,stroke:#333

style Q1 fill:#ffcccb,stroke:#333

style Q2 fill:#ffcccb,stroke:#333

style Q3 fill:#ffcccb,stroke:#333

3.12.2 Anti-Pattern Checklist

Anti-Pattern

What It Sounds Like

What Happens Next

"Agile means no docs"

"We'll document it after we build it"

Documentation never happens; tribal knowledge

Anti-Pattern	What It Sounds Like	What Happens Next
"The developer knows"	"Ask Sarah, she built it"	Sarah leaves; no one knows how it works
"Source is golden"	"We trust the source system"	Nulls, duplicates, and type mismatches in production
"Recon is optional"	"Row counts are close enough"	\$2M variance discovered 6 months later
"One big story"	"Epic: Build Customer Pipeline"	Unestimatable; never "done"
"Copy-paste mapping"	"It's like the other pipeline"	Subtle differences cause silent failures

3.12.3 Recovery Strategies

If You're Here...	Do This
No DRD exists	Stop. Write one now. 2 hours saves 2 weeks.
DMD is in someone's head	Schedule a "brain dump" session; document as you go
No DQ rules	Start with just NOT NULL on keys; add rules incrementally
Stories are too big	Split by layer: Bronze, Silver, Gold as separate stories
Recon is failing	Add logging; compare row counts daily until stable

3.13 Complete Example: Customer 360 Pipeline

3.13.1 Business Request

markdown

Business Request: Customer 360 View

Objective

Create a unified customer view combining CRM data with order history to support marketing segmentation and lifetime value analysis.

Requirements

- Daily refresh, available by 6 AM
- Include customer demographics, order history, calculated LTV
- Support BI dashboards and ML model training
- Handle PII appropriately (GDPR compliance)

Source Systems

- CRM (PostgreSQL): customers, addresses
- Order Management (MySQL): orders, order_items

Consumers

- Marketing Analytics team (Tableau dashboards)
- Data Science team (Feature store)
- Customer Success dashboards (Looker)

3.13.2 Artifact Flow for Customer 360

mermaid

flowchart TD

subgraph "Phase 1: Requirements"

BR[Business Request:
Customer 360] --> DA[Data Analyst]

DA --> DRD[DRD:
Sources, Rules, SLAs]

end

subgraph "Phase 2: Design"

DRD --> ARCH[Data Architect]

ARCH --> HLD[HLD:
Medallion Architecture]

HLD --> MOD[Data Modeler]

MOD --> SCHEMAS[Schemas:
Bronze/Silver/Gold]

end

subgraph "Phase 3: Specification"

SCHEMAS --> MAP[Mapping Engineer]

DRD --> MAP

MAP --> DMD[DMD:
Column Mappings]

DMD --> DQE[DQ Engineer]

DQE --> DQS[DQS:
Quality Rules]

end

subgraph "Phase 4: Implementation"

DQS --> TL[Technical Lead]

HLD --> TL

TL --> LLD[LLD:
DAGs + Tasks]

LLD --> SM[Scrum Master]

SM --> ISSUES[GitHub Issues:
8 Stories]

end

style BR fill:#e8e8e8,stroke:#333

style DRD fill:#a8e6cf,stroke:#333

style HLD fill:#dcedc1,stroke:#333

style SCHEMAS fill:#ffd3a5,stroke:#333

style DMD fill:#ffaaa5,stroke:#333

style DQS fill:#ff8b94,stroke:#333

style LLD fill:#d4a5ff,stroke:#333

style ISSUES fill:#a5d4ff,stroke:#333

3.13.3 Cross-References Between Documents

mermaid

flowchart LR

subgraph DRD["DRD"]

DRD_BR[BR001: LTV Calculation]

DRD_DQ[DQ: customer_id NOT NULL]

end

subgraph DMD["DMD"]

DMD_COL[lifetime_value column]

DMD_REF["Business Rule: BR001"]

end

subgraph DQS["DQS"]

DQS_RULE[GLD-003: LTV >= 0]

DQS_REF["Source: DRD DQ requirements"]

end

subgraph LLD["LLD"]

LLD_TASK[Task: calculate_ltv]

LLD_REF["Mapping: DMD Section 3.3"]

end

subgraph Story["Story 1.6"]

ST_AC["AC: LTV calculated per BR001"]

ST_DQ["DQ: GLD-003 passes"]

end

DRD_BR --> DMD_REF

DMD_COL --> DQS_RULE

DMD_COL --> LLD_REF

DRD_DQ --> DQS_REF

DMD_REF --> ST_AC

DQS_RULE --> ST_DQ

style DRD fill:#a8e6cf,stroke:#333

style DMD fill:#ffaaa5,stroke:#333

style DQS fill:#ff8b94,stroke:#333

style LLD fill:#d4a5ff,stroke:#333

style Story fill:#a5d4ff,stroke:#333

Part IV: From Human Roles to AI Agents

3.14 The Agent Design Pattern

3.14.1 Why Automate Planning?

Manual Planning	Agent-Assisted Planning
Inconsistent artifact quality	Template-driven consistency
Easy to miss edge cases	Systematic coverage
Time-consuming documentation	Rapid first drafts
Knowledge in people's heads	Codified in prompts
Single person bottleneck	Parallelizable

What agents do well:

- Generate comprehensive first drafts
- Apply templates consistently
- Cross-reference between artifacts
- Flag missing information
- Ensure completeness (all columns mapped, all rules defined)

Where humans must decide:

- Business rule correctness
- Architecture trade-offs
- Priority and sequencing
- Risk assessment
- Organizational politics

3.14.2 When Agents Struggle

Be realistic about limitations:

Scenario	Agent Challenge	Human Required For
Ambiguous business rules	Cannot infer unstated intent	Clarification with stakeholders
Novel architecture needs	Pattern matching fails	Creative design decisions
Organizational constraints	No context about politics	Stakeholder navigation
Performance tuning	Needs runtime profiling data	Actual performance testing
Trade-off decisions	Can list options, not choose	Business judgment
Legacy system quirks	Not in documentation	Tribal knowledge

 The Human-Agent Contract

Agents generate; humans validate. Agents are fast and thorough; humans are wise and contextual. Never deploy agent-generated artifacts without human review.

3.14.3 The Modular Agent Architecture



flowchart TD

subgraph "Agent Module"

INS[instructions.md
————
Role persona
Responsibilities
Output format]
TOOLS[tools.yaml
————
Available tools
for this agent]
SKILLS[skills.yaml
————
Capabilities
enabled]
CONN[connections.yaml
————
Data sources
accessible]

end

subgraph "Orchestrator"

ORCH[Load Agent Module]
EXEC[Execute with Context]
REVIEW[Human Review Gate]

end

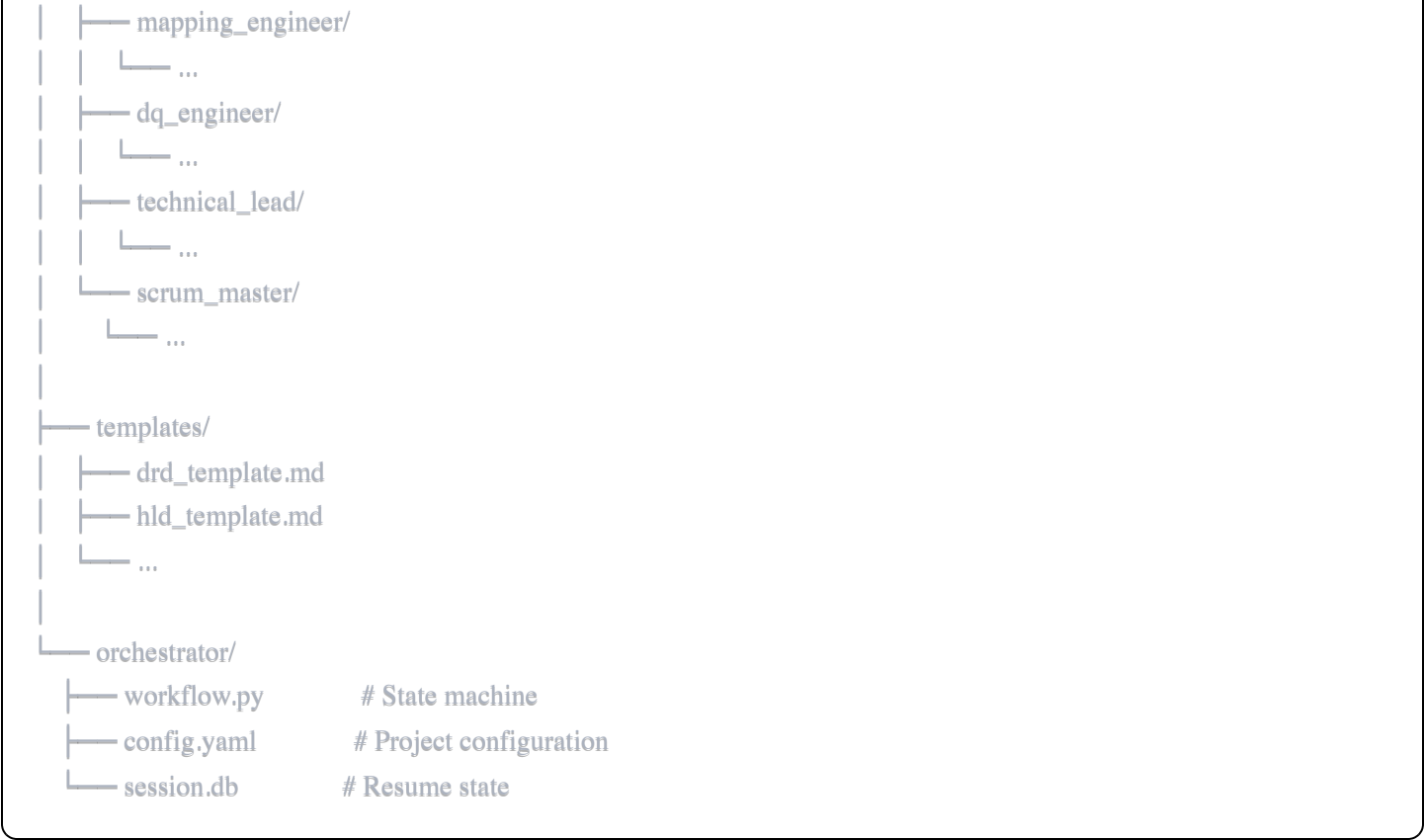
INS --> ORCH
TOOLS --> ORCH
SKILLS --> ORCH
CONN --> ORCH
ORCH --> EXEC
EXEC --> REVIEW
REVIEW -->|Approved| NEXT[Next Agent]
REVIEW -->|Feedback| EXEC

style INS fill:#4ecdc4,stroke:#333
style TOOLS fill:#ffd93d,stroke:#333
style SKILLS fill:#6bc977,stroke:#333
style CONN fill:#ff6b6b,stroke:#333
style REVIEW fill:#ffd700,stroke:#333

3.14.4 Agent Directory Structure

planning-with-intent/

```
|— agents/
| |— data_analyst/
| | |— instructions.md    # Role definition & output format
| | |— tools.yaml        # document_reader, catalog_browser
| | |— skills.yaml        # requirements_extraction
| | |— connections.yaml   # source_systems, data_catalog
| |
| |— data_architect/
| |   ...
| |— data_modeler/
| |   ...
```

3.14.5 Agent Execution Flow



sequenceDiagram

participant User

participant Orchestrator

participant Agent

participant Tools

participant Output

User->>Orchestrator: Start planning workflow

loop For each agent in sequence

Orchestrator->>Orchestrator: Load agent module

Orchestrator->>Agent: Initialize with context

Agent->>Tools: Use available tools

Tools-->>Agent: Tool results

Agent->>Output: Generate artifact

Output->>User: Present for review

alt Approved

User->>Orchestrator: Approve & continue

else Feedback

User->>Agent: Provide feedback

Agent->>Output: Revise artifact

end

end

Orchestrator->>User: Workflow complete

3.15 Agent Prompt Evolution: From Messy to Refined

3.15.1 Why This Matters

Agent prompts (the `instructions.md` files) require iteration. Here's how a Data Analyst agent prompt evolved through testing:

3.15.2 First Draft (Too Vague)

markdown

Data Analyst Agent - Version 1 (PROBLEMATIC)

You are a data analyst. Analyze the business request and create a requirements document.

Think about what sources are needed and what the business wants.

Problems:

- No structure → inconsistent outputs
- No output format → can't parse results
- No clarification protocol → makes assumptions

3.15.3 Second Draft (Over-Specified)

markdown

Data Analyst Agent - Version 2 (PROBLEMATIC)

You are a Senior Data Analyst with 15 years of experience in retail, finance, and healthcare domains. You have expertise in PostgreSQL, MySQL, Oracle, SQL Server, MongoDB, Cassandra...

[500 more words of background]

You MUST follow these 47 steps:

1. Read the document
2. Identify the first noun
3. Check if the noun is a system name
4. If yes, add to source list
5. If no, check if it's a table name

...

Problems:

- Too rigid → breaks on edge cases
- Excessive persona → dilutes key instructions
- Step-by-step micro-management → agent can't adapt

3.15.4 Final Version (Balanced)

markdown

Data Analyst Agent

Role

You are a Senior Data Analyst specializing in requirements gathering for data engineering projects.

Objective

Analyze business requests to produce a comprehensive Data Requirements Document (DRD) that captures:

1. Source systems and their characteristics
2. Business rules and transformations
3. Data quality expectations
4. Consumer requirements and SLAs

Process

Step 1: Source Discovery

For each data source mentioned or implied:

- Identify system name and type
- Document access method
- Estimate data volumes
- Note CDC capabilities

Step 2: Business Rule Extraction

Identify and document:

- Calculations and derivations
- Conditional logic
- Default values

Step 3: Quality Requirements

Determine:

- Mandatory fields
- Valid value ranges
- Referential integrity needs

Step 4: Consumer Analysis

Understand:

- Who consumes this data
- What SLAs are required

Output Format

Generate a DRD following the template at:

``templates/drd_template.md``

Clarification Protocol

If information is missing, list specific questions at the end of the document under "Open Questions" rather than making assumptions.

Why this works:

- Clear role without excessive backstory
- Structured process without micro-management
- Explicit output format for consistency
- Clarification protocol prevents hallucination

3.15.5 Prompt Engineering Principles

Principle	Example
Be specific about outputs	"Generate a DRD following template X"
Provide structure, not scripts	"Step 1: Source Discovery" not "1. Find first noun"
Include escape hatches	"List under Open Questions rather than assuming"
Reference external files	Templates, schemas, examples
Test with edge cases	Ambiguous requests, missing info

3.16 Agent: Business/Data Analyst

3.16.1 instructions.md

markdown

Data Analyst Agent

Role

You are a Senior Data Analyst specializing in requirements gathering for data engineering projects.

Objective

Analyze business requests to produce a comprehensive Data Requirements Document (DRD) that captures:

1. Source systems and their characteristics
2. Business rules and transformations
3. Data quality expectations
4. Consumer requirements and SLAs

Process

Step 1: Source Discovery

For each data source mentioned or implied:

- Identify system name and type
- Document access method
- Estimate data volumes
- Note CDC capabilities

Step 2: Business Rule Extraction

Identify and document:

- Calculations and derivations
- Conditional logic
- Default values
- Data standardizations

Step 3: Quality Requirements

Determine:

- Mandatory fields
- Valid value ranges
- Referential integrity needs
- Tolerance thresholds

Step 4: Consumer Analysis

Understand:

- Who consumes this data
- What SLAs are required
- What freshness is needed

Output Format

Generate a DRD following the template at:

`templates/drd\_template.md`

Clarification Protocol

If information is missing, list specific questions at the end

of the document under "Open Questions" rather than making assumptions.

3.16.2 tools.yaml

yaml

tools:

- **name:** document_reader
description: Read and parse input documents
- **name:** catalog_browser
description: Browse existing data catalog for source metadata
- **name:** template_filler
description: Fill DRD template with extracted information
- **name:** question_generator
description: Generate clarifying questions for missing info

3.16.3 skills.yaml

yaml

skills:

- **name:** requirements_extraction
description: Extract structured requirements from unstructured text
- **name:** source_system_analysis
description: Analyze source system characteristics
- **name:** business_rule_documentation
description: Document business rules in standard format

3.16.4 connections.yaml

yaml

connections:

- **name:** source_catalog

type: data_catalog

description: Existing data catalog for source system metadata

- **name:** business_glossary

type: reference

description: Business term definitions

3.17 Agent: Data Architect

3.17.1 instructions.md

markdown

Data Architect Agent

Role

You are a Senior Data Architect responsible for designing scalable, maintainable data pipeline architectures.

Objective

Create a High-Level Design (HLD) document that defines:

1. Architecture pattern and justification
2. Layer specifications
3. Technology stack recommendations
4. CDC strategy
5. Capacity planning

Process

Step 1: Requirements Analysis

Review the DRD to understand:

- Data volumes and growth projections
- Latency requirements
- Compliance needs
- Consumer patterns

Step 2: Pattern Selection

Select and justify architecture pattern:

- Medallion: Standard batch/micro-batch
- Lambda: Real-time + batch required
- Kappa: Streaming-first
- Data Vault: Audit/compliance critical

Step 3: Technology Evaluation

Recommend technology for each component:

- Orchestration
- Processing engine
- Storage format
- Quality framework

Step 4: CDC Strategy

Document change capture approach:

- CDC method per source
- Schema evolution handling
- Fallback strategies

Step 5: Capacity Planning

Project resource needs:

- Current volumes
- Growth projections
- Cost estimates

Output Format

Generate an HLD following the template at:

`templates/hld\_template.md`

Decision Documentation

For each major decision, include:

- Options considered
- Selection rationale
- Trade-offs accepted

3.17.2 tools.yaml

yaml

tools:

- **name:** diagram_generator
description: Generate architecture diagrams in Mermaid format
- **name:** template_filler
description: Fill HLD template with design decisions
- **name:** tech_evaluator
description: Compare technology options against requirements
- **name:** capacity_calculator
description: Estimate compute and storage requirements

3.18 Agent: Data Modeler

(If separate from Architect)

3.18.1 instructions.md

markdown

Data Modeler Agent

Role

You are a Data Modeler specializing in dimensional modeling and multi-layer schema design for data platforms.

Objective

Create Data Model Specifications that define:

1. Bronze layer schemas (source-aligned)
2. Silver layer schemas (canonical entities)
3. Gold layer schemas (dimensional model)

Process

Step 1: Bronze Layer Design

For each source entity:

- Create source-aligned schema
- Add metadata columns (_ingestion_ts, _source_file, etc.)
- Define partition strategy

Step 2: Silver Layer Design

For each business entity:

- Design canonical schema
- Define primary/foreign keys
- Apply naming conventions

Step 3: Gold Layer Design

For analytical models:

- Design dimension tables (with SCD type)
- Define fact tables with grain
- Create aggregate tables if needed

Output Format

Generate specifications following the template at:

`templates/data\_model\_template.md`

3.19 Agent: Mapping Engineer

3.19.1 instructions.md

markdown

Mapping Engineer Agent

Role

You are a Data Mapping Specialist responsible for creating detailed source-to-target mappings with transformation logic.

Objective

Create a Data Mapping Document (DMD) that specifies:

1. Column-level mappings for each pipeline stage
2. Transformation logic
3. Null handling strategies
4. Business rule implementations
5. End-to-end lineage
6. Edge case handling

Process

Step 1: Source → Bronze Mappings

For each source table:

- Map source columns to bronze columns
- Define type conversions
- Add metadata columns

Step 2: Bronze → Silver Mappings

For each entity:

- Map bronze to silver columns
- Define cleansing transformations
- Implement business rules
- Specify null handling

Step 3: Silver → Gold Mappings

For each target table:

- Map silver to gold columns
- Define aggregations and joins
- Implement SCD logic

Step 4: Edge Case Documentation

Document handling for:

- Source column changes
- Type mismatches
- Null in required fields
- Duplicate keys
- Late-arriving data

Step 5: Lineage Documentation

Create end-to-end lineage showing column flow.

Output Format

Generate a DMD following the template at:

``templates/dmd_template.md``

Completeness Check

Ensure every target column has:

- Source column(s) identified
- Transformation specified
- Null handling defined

3.20 Agent: Data Quality Engineer

3.20.1 instructions.md

markdown

Data Quality Engineer Agent

Role

You are a Data Quality Engineer responsible for defining comprehensive quality rules and reconciliation specifications.

Objective

Create a Data Quality Specification (DQS) that defines:

1. Column-level validation rules
2. Cross-table validations
3. Reconciliation specifications
4. Anomaly detection rules

Process

Step 1: Rule Derivation

From DRD and DMD, derive rules for:

- Completeness (NOT NULL)
- Uniqueness (duplicates)
- Validity (format, range)
- Consistency (cross-field)
- Referential integrity

Step 2: Threshold Definition

For each rule, define:

- Pass threshold (e.g., 99.9%)
- Severity (critical/warning/info)
- Action on failure

Step 3: Reconciliation Design

Define source-to-target reconciliation:

- Row count comparisons
- Sum validations
- Key matching

Output Format

Generate a DQS following the template at:

``templates/dqs_template.md``

3.21 Agent: Technical Lead

3.21.1 instructions.md

markdown

Technical Lead Agent

Role

You are a Technical Lead responsible for creating implementation plans and Low-Level Designs for data pipelines.

Objective

Create a Low-Level Design (LLD) that defines:

1. DAG architecture and task structure
2. Error handling strategies
3. Performance optimizations
4. Implementation sequence

Process

Step 1: DAG Design

Define:

- Master DAG structure
- Task dependencies
- Parallelization opportunities
- Scheduling configuration

Step 2: Task Specifications

For each task:

- Input/output datasets
- Processing logic reference (DMD)
- DQ checks to execute
- Retry configuration

Step 3: Error Handling

Define:

- Retry strategies per error type
- Dead letter queue design
- Alert routing
- Recovery procedures

Step 4: Implementation Sequence

Create phased plan:

- Phase 1: Infrastructure
- Phase 2: Bronze layer
- Phase 3: Silver layer
- Phase 4: Gold layer
- Phase 5: Integration

Output Format

Generate an LLD following the template at:

`templates/lld\_template.md`

3.22 Agent: Scrum Master / Story Writer

3.22.1 instructions.md

markdown

Scrum Master Agent

Role

You are a Scrum Master / Delivery Lead responsible for translating technical plans into actionable work items.

Objective

Create Epics and Stories that:

1. Break work into sprint-sized chunks
2. Reference source artifacts
3. Include clear acceptance criteria
4. Enable accurate estimation

Process

Step 1: Epic Creation

For each major deliverable:

- Create epic with business value
- Define scope and boundaries
- List dependencies
- Identify risks

Step 2: Story Decomposition

For each epic, break into stories following INVEST:

- Independent
- Negotiable
- Valuable
- Estimable
- Small
- Testable

Step 3: Artifact Linking

For each story:

- Reference relevant DMD sections
- Link to DQ rules
- Include LLD task references

Step 4: Issue Creation

Generate GitHub Issues / Jira tickets with proper labels and assignments.

Output Format

Generate epics/stories following templates at:

- `templates/epic\_template.md`
- `templates/story\_template.md`

Labels

Apply appropriate labels:

- type: ingestion, transformation, dq, orchestration
- layer: bronze, silver, gold
- priority: p0, p1, p2

Part V: Orchestrating the Agent Pipeline

3.23 The Planning-with-Intent (PWI) Orchestrator

What does PWI stand for?

Planning With Intent — a framework for intentional, structured planning of data engineering projects.

3.23.1 State Machine Design

mermaid

stateDiagram-v2

[*] --> Init

Init --> DataAnalyst: Start workflow

DataAnalyst --> AnalystReview: DRD generated

AnalystReview --> DataArchitect: Approved

AnalystReview --> DataAnalyst: Feedback

DataArchitect --> ArchitectReview: HLD generated

ArchitectReview --> DataModeler: Approved

ArchitectReview --> DataArchitect: Feedback

DataModeler --> ModelerReview: Schemas generated

ModelerReview --> MappingEngineer: Approved

ModelerReview --> DataModeler: Feedback

MappingEngineer --> MappingReview: DMD generated

MappingReview --> DQEngineer: Approved

MappingReview --> MappingEngineer: Feedback

DQEngineer --> DQReview: DQS generated

DQReview --> TechnicalLead: Approved

DQReview --> DQEngineer: Feedback

TechnicalLead --> TechLeadReview: LLD generated

TechLeadReview --> ScrumMaster: Approved

TechLeadReview --> TechnicalLead: Feedback

ScrumMaster --> StoryReview: Stories generated

StoryReview --> Sync: Approved

StoryReview --> ScrumMaster: Feedback

Sync --> Complete: Issues created

Complete --> [*]

DataAnalyst --> Paused: User pause

DataArchitect --> Paused: User pause

MappingEngineer --> Paused: User pause

DQEngineer --> Paused: User pause

TechnicalLead --> Paused: User pause

ScrumMaster --> Paused: User pause

Paused --> DataAnalyst: Resume
Paused --> DataArchitect: Resume
Paused --> MappingEngineer: Resume
Paused --> DQEngineer: Resume
Paused --> TechnicalLead: Resume
Paused --> ScrumMaster: Resume

3.23.2 Configuration Options

yaml

```

# config.yaml

project:
  name: "Customer 360 Pipeline"
  type: data_engineering

workflow:
  # Enable/disable optional agents
  agents:
    data_analyst: true
    data_architect: true
    data_modeler: true    # Set false if architect handles modeling
    mapping_engineer: true
    dq_engineer: true
    technical_lead: true
    scrum_master: true

  # Review gate behavior
  review_gates:
    mode: interactive    # interactive / auto-approve / batch
    timeout_minutes: 60

  # Resume capability
  persistence:
    enabled: true
    backend: sqlite      # sqlite / json
    path: ./sessions/

  # Output configuration
  output:
    artifacts_dir: ./docs/planning/
    issues:
      backend: github    # github / jira
      repo: "org/repo"
      labels:
        - planning-with-intent
        - auto-generated

```

3.23.3 Resume Capability

mermaid

flowchart TD

START[Start Workflow] --> CHECK{Session
exists?}

CHECK -->|No| NEW[Create new session]

CHECK -->|Yes| LOAD[Load session state]

NEW --> INIT[Initialize at first agent]

LOAD --> RESUME[Resume at last checkpoint]

INIT --> RUN[Run workflow]

RESUME --> RUN

RUN --> SAVE[Save state after each phase]

SAVE --> GATE{Review
gate}

GATE -->|Approved| NEXT[Next agent]

GATE -->|Pause| PERSIST[Persist state]

GATE -->|Feedback| REVISE[Revise artifact]

NEXT --> RUN

REVISE --> RUN

PERSIST --> EXIT[Exit workflow]

EXIT --> LATER[User returns later]

LATER --> CHECK

style CHECK fill:#ffd700,stroke:#333

style GATE fill:#ffd700,stroke:#333

style PERSIST fill:#4ecdc4,stroke:#333

3.24 Hands-On: Your First Agent Pipeline

3.24.1 Prerequisites

- ✓ Claude Pro or Max subscription
- ✓ Docker installed and running
- ✓ Python 3.11+
- ✓ Git
- ✓ GitHub account with personal access token

3.24.2 Installation

```
bash

# Clone the repository
git clone https://github.com/yourorg/planning-with-intent.git
cd planning-with-intent

# Create virtual environment
python -m venv venv
source venv/bin/activate # or `venv\Scripts\activate` on Windows

# Install dependencies
pip install -e .

# Start the LiteLLM proxy (for Claude authentication)
docker-compose up -d litellm-proxy

# Configure your environment
cp .env.example .env
# Edit .env with your Claude session token and GitHub PAT
```

3.24.3 Running the Workflow

```
bash

# Initialize a new project
pwi init --name "Customer 360" --type data_engineering

# Start the planning workflow
pwi plan ./examples/customer-360-request.md

# The orchestrator will:
# 1. Run Data Analyst agent → generate DRD
# 2. Pause for your review
# 3. On approval, run Data Architect → generate HLD
# 4. ... continue through all agents
```

3.24.4 Review Process

```
mermaid
```


sequenceDiagram

participant CLI

participant Orchestrator

participant Agent

participant User

participant Editor

CLI->>Orchestrator: pwi plan request.md

Orchestrator->>Agent: Execute Data Analyst

Agent-->>Orchestrator: DRD generated

Orchestrator->>User: DRD ready for review

User->>Editor: Review artifact

Editor-->>User: View/edit

alt Approve

User->>CLI: pwi approve

CLI->>Orchestrator: Continue workflow

else Feedback

User->>CLI: pwi feedback "Need more detail on BR003"

CLI->>Agent: Revise with feedback

Agent-->>Orchestrator: Updated DRD

else Pause

User->>CLI: pwi pause

Orchestrator->>Orchestrator: Save state

end

3.24.5 CLI Commands Reference

bash

```
# Core commands
pwi init --name "Project" --type data_engineering
pwi plan <request.md>
pwi approve          # Approve current artifact
pwi feedback "Your feedback" # Request revisions
pwi pause            # Save and exit
pwi resume           # Continue from checkpoint
pwi status           # Show current state

# Artifact generation
pwi generate drd      # Generate just DRD
pwi generate hld      # Generate just HLD
pwi generate dmd --entity customers
pwi generate dqs --pipeline customer_dimension

# Validation
pwi validate dmd docs/planning/dmd.md
pwi validate dqs docs/planning/dqs.md

# Export
pwi export github     # Push issues to GitHub
pwi export jira       # Push issues to Jira
```

3.24.6 Final Output

After completing the workflow, you'll have:

```
docs/planning/
├── drd.md           # Data Requirements Document
├── hld.md           # High-Level Design
├── data-models/
│   ├── bronze_schemas.md
│   ├── silver_schemas.md
│   └── gold_schemas.md
├── dmd.md           # Data Mapping Document
├── dqs.md           # Data Quality Specification
├── lld.md           # Low-Level Design
└── stories/
    ├── epic-1.md
    ├── story-1-1.md
    ├── story-1-2.md
    └── ...
```

Plus GitHub Issues created and labeled

Appendices

Appendix A: Template Reference

A.1 Data Requirements Document (DRD) Template

Full template in companion repository: `templates/drd_template.md`

A.2 High-Level Design (HLD) Template

Full template in companion repository: `templates/hld_template.md`

A.3 Data Model Specification Template

Full template in companion repository: `templates/data_model_template.md`

A.4 Data Mapping Document (DMD) Template

Full template in companion repository: `templates/dmd_template.md`

A.5 Data Quality Specification (DQS) Template

Full template in companion repository: `templates/dqs_template.md`

A.6 Low-Level Design (LLD) Template

Full template in companion repository: [templates/lld_template.md](#)

A.7 Epic Template

Full template in companion repository: [templates/epic_template.md](#)

A.8 Story Template

Full template in companion repository: [templates/story_template.md](#)

Appendix B: Agent Instructions Reference

All agent instruction files are available in the companion repository under [agents/*/instructions.md](#).

Appendix C: Complete Customer 360 Artifact Set

The complete worked example is available in the companion repository under [examples/customer-360/](#).

Appendix D: Glossary

Term	Also Known As	Definition
DRD	Data Spec, Requirements Spec	Document capturing data requirements
HLD	Architecture Doc	High-level technical design
DMD	Mapping Spec, ETL Spec, S2T	Source-to-target column mappings
DQS	DQ Rules, Validation Spec	Data quality rule definitions
LLD	Technical Spec	Detailed implementation design
SCD	Slowly Changing Dimension	Dimension history tracking pattern
CDC	Change Data Capture	Real-time change tracking
Bronze	Raw, Landing	First layer in medallion architecture
Silver	Cleansed, Conformed	Second layer in medallion architecture

Term	Also Known As	Definition
Gold	Curated, Business	Final layer in medallion architecture
INVEST	-	Story quality criteria: Independent, Negotiable, Valuable, Estimable, Small, Testable
PWI	Planning-with-Intent	This framework

Summary: Chapter Flow

mermaid

flowchart TD

subgraph "THE HUMAN WORKFLOW"

subgraph "Part I: The Problem"

P1[3.1 Why Projects Fail]

P2[3.2 The Artifact Chain]

end

subgraph "Part II: Human Roles"

R0[3.3 Role Overview]

R1[3.4 Data Analyst]

R2[3.5 Data Architect]

R3[3.6 Data Modeler]

R4[3.7 Mapping Engineer]

R5[3.8 DQ Engineer]

R6[3.9 Technical Lead]

R7[3.10 Scrum Master]

end

subgraph "Part III: Artifact Flow"

F1[3.11 Dependencies]

F2[3.12 Warning Signs]

F3[3.13 Complete Example]

end

end

subgraph "AGENT AUTOMATION"

subgraph "Part IV: Agents"

A0[3.14 Agent Design]

A1[3.15 Prompt Evolution]

A2[3.16-3.22 Agent Definitions]

end

subgraph "Part V: Orchestration"

O1[3.23 PWI Orchestrator]

O2[3.24 Hands-On Tutorial]

end

end

P1 --> P2 --> R0

R0 --> R1 --> R2 --> R3 --> R4 --> R5 --> R6 --> R7

R7 --> F1 --> F2 --> F3

F3 --> A0 --> A1 --> A2

A2 --> O1 --> O2

```
style P1 fill:#ffcccb,stroke:#333
style P2 fill:#ffcccb,stroke:#333
style R1 fill:#a8e6cf,stroke:#333
style R2 fill:#dcedc1,stroke:#333
style R3 fill:#ffd3a5,stroke:#333
style R4 fill:#ffaaa5,stroke:#333
style R5 fill:#ff8b94,stroke:#333
style R6 fill:#d4a5ff,stroke:#333
style R7 fill:#a5d4ff,stroke:#333
style F2 fill:#ffcccb,stroke:#333
style A0 fill:#4ecdc4,stroke:#333
style A1 fill:#4ecdc4,stroke:#333
style O2 fill:#6bc777,stroke:#333
```

End of Chapter 3: Planning with Intent for Data Engineering